VAUTRIN OS

Codebase Review & Updated Specification Pack - v0.2

Implementation snapshot reviewed: vautrin-os-v-01

Codebase Review

Vautrin OS Codebase Review - v0.2

Review scope

This review compares the implementation in `vautrin-os-v-01` with the V0.1 SDDs, ADRs, AGENTS.md, and technical backlog.

The codebase is still foundation-heavy, which is appropriate at this stage. The strongest part of the implementation is the security-oriented multi-tenant foundation. The largest immediate risks are in refresh-token concurrency, active-tenant continuity during refresh, specification drift, and moving into business-domain features before closing the authentication edge cases.

What is implemented well

1. Monorepo foundation matches the architecture

The repository follows the intended pnpm + Turborepo structure with separate `web`, `api`, and `worker` applications and shared packages. Build artifacts and dependencies are ignored by Git rather than tracked.

2. Multi-tenancy is treated as a security boundary

The implementation goes materially beyond a simple `tenant_id` filter:

• runtime and migration database roles are separated;

• tenant-scoped tables use PostgreSQL RLS;

• `FORCE ROW LEVEL SECURITY` is enabled;

• transaction-local `set_config` avoids tenant context leaking through pooled connections;

• an RLS coverage check guards against future tenant-scoped tables being added without policies;

• integration tests exercise cross-tenant SELECT/INSERT/UPDATE/DELETE and vector isolation.

This is aligned with ADR-003 and ADR-005 and is a strong foundation for the SaaS model.

3. Authentication design is substantially aligned with ADR-013

The code implements:

• Argon2id password verification;

• short-lived JWT access tokens;

• opaque refresh tokens;

• SHA-256 refresh-token digests at rest;

• refresh-token families;

• rotation and reuse detection;

• HttpOnly refresh cookies;

• active tenant switching with membership re-verification;

• capability-based RBAC.

4. CI/security posture is ahead of typical MVP quality

The repository includes linting, typechecking, unit/integration jobs, RLS assertions, Semgrep, dependency auditing, secret scanning, CodeQL configuration, and non-root container checks.

5. The code explains architectural intent

Comments consistently reference SDDs/ADRs and explain security-sensitive decisions. This is especially useful when coding agents participate in implementation.

Findings requiring correction

F-01 - P0 - Refresh rotation has a concurrency race

`RefreshTokenService.rotate()` first reads the token as valid and later performs an unconditional update inside a transaction. Two concurrent requests can both observe the predecessor as unused, both create successors, and both rotate successfully.

This violates the intended one-time-use property of refresh rotation.

Required correction: make token consumption atomic. The write that claims the predecessor must condition on `revoked_at IS NULL` and `replaced_by_id IS NULL` (or use an equivalent row-lock/serializable strategy). Exactly one request may succeed. A losing request should trigger the defined reuse policy.

Add a concurrency integration test that submits the same refresh token twice simultaneously and asserts that at most one successor becomes usable.

F-02 - P0 - Active tenant is lost after refresh for multi-tenant users

`switchTenant()` issues an access token with a selected tenant, but the refresh token/session does not preserve that selection. `refresh()` chooses an active tenant only when the user has exactly one membership. A user with multiple memberships therefore becomes tenant-less after the access token expires.

Required correction: persist `active_tenant_id` in the server-side refresh session/token record, or require a requested tenant during refresh and re-verify membership. The preferred V0.2 design is to persist the active tenant on the refresh session and update it when tenant switching occurs.

F-03 - P0 - Membership revocation is not an atomic security operation

`TenantService.revokeMembership()` changes membership state but only documents that refresh tokens must also be revoked. The method does not enforce that behavior itself.

Required correction: create an application use case for membership revocation that performs membership revocation plus relevant session-family revocation and audit. Do not expose the lower-level membership mutation directly as the complete security operation.

The 15-minute access-token stale window remains an explicit ADR trade-off unless sensitive routes add membership revalidation.

F-04 - P1 - Authentication endpoints need abuse controls

The implementation protects against account enumeration but there is no documented rate-limit/lockout strategy for login and refresh endpoints.

Required correction: add rate limiting keyed by IP and normalized account identifier where appropriate, with observability and without creating a trivial account-lockout DoS.

F-05 - P1 - Cookie security needs an explicit deployment/CSRF contract

The refresh token is correctly HttpOnly/Secure in production and `SameSite=Lax`, but the architecture does not yet define trusted origins, reverse-proxy behavior, CSRF handling, or the exact production host topology.

Required correction: document same-origin deployment as the preferred topology. Add trusted-origin checks/CSRF protection for cookie-authenticated state-changing endpoints as required by the final topology. Configure trusted proxy behavior before treating `request.ip` as the original client IP.

F-06 - P1 - Specification drift already exists

The database SDD says its table set is exact, but the implementation added `refresh_tokens` through ADR-013. The backlog also says the database package contains four migrations while the repository contains six. This is expected during development, but the docs must remain the source of truth if coding agents are instructed to follow them.

Required correction: update SDD-03, SDD-04, SDD-09, ADR-013, and backlog metadata as part of this review pack.

F-07 - P1 - Service and service-area modeling must be decided before E4

The backlog requires Services and Service Areas, but the current core schema has no dedicated `Service` or `ServiceArea` model. `Location.serviceRadiusMiles` is not sufficient for contractors serving multiple cities/ZIP codes, and services should not become arbitrary unvalidated JSON if they are used by qualification, analytics, and AI tools.

Decision for V0.2: add tenant-scoped `services` and `service_areas` tables before implementing VOS-042/VOS-043. Keep vertical templates responsible for seeding them.

F-08 - P1 - Audit coverage is intentionally partial and should not be marked complete

Authentication/authorization audit behavior exists, but customer mutations, workflow activation, AI tools, and integration actions are not implemented yet. Keep VOS-121 IN_PROGRESS until those surfaces exist.

F-09 - P2 - Global identity access deserves tighter infrastructure boundaries later

The runtime database role can access global identity/session tables because login occurs before tenant selection. Current code intentionally limits unscoped access through `DatabaseService`, but this remains an application-level convention for global tables.

For MVP this is acceptable. Before broader team access or plugin-like extensions, consider narrower database privileges or dedicated database functions/repositories for identity/session operations.

Verification limitation

The repository was inspected directly. I could not execute the pnpm test suite in the review environment because the required pinned pnpm binary was not locally available and outbound package-registry access was unavailable. The repository itself contains CI definitions and generated build/test artifacts indicating active validation, but this review does not claim an independently executed green test run.

Overall assessment

The implementation is directionally strong and consistent with the intended architecture. Do not start AI/workflow implementation yet. Close F-01 through F-06, then implement Business Configuration and CRM as the next product slice.

Next Implementation Plan

Vautrin OS - Next Implementation Plan

Gate A - Close authentication/security foundation

VOS-037 [P0] Atomic refresh-token rotation

• Make predecessor consumption atomic.

• Add concurrent refresh integration test.

• Assert only one successor can be redeemed.

• Preserve reuse-detection semantics.

VOS-038 [P0] Preserve active tenant across refresh

• Add `active_tenant_id` to the refresh-session model.

• Verify membership before persisting/switching it.

• Rotate the selected tenant into successor refresh records.

• Add multi-membership refresh E2E coverage.

VOS-039 [P0] Secure membership revocation use case

• Revoke membership.

• Revoke affected refresh sessions/families.

• Audit the action.

• Add integration coverage.

VOS-027 [P1] Authentication abuse controls

• Rate-limit login and refresh.

• Add metrics/logging for throttling.

• Avoid permanent account lockout from unauthenticated traffic.

VOS-028 [P1] Cookie/origin/proxy security contract

• Define production web/API host topology.

• Configure trusted proxy behavior.

• Define trusted Origin policy and CSRF strategy for cookie-authenticated endpoints.

Gate B - Reconcile architecture documents

VOS-014 [P0] Documentation consistency gate

• Update SDD-03 with RefreshToken/session entity.

• Update SDD-04 table inventory and migration expectations.

• Update SDD-09 authentication mechanism.

• Update ADR-013 with active-tenant persistence and atomic rotation requirements.

• Add ADR-014 for Service/ServiceArea modeling.

• Update backlog statuses after each merged PR.

Gate C - Business Configuration (next product milestone)

VOS-040 [P1] Business profile API

VOS-041 [P1] Locations API

VOS-042 [P1] Services domain + API

VOS-043 [P1] Service areas domain + API

VOS-044 [P1] Business hours

VOS-045 [P1] Custom field definitions

VOS-046 [P1] Custom field values

Milestone acceptance: a remodeling tenant can be provisioned and fully configured without direct database edits.

Gate D - CRM vertical slice

VOS-050 [P1] Contacts

VOS-051 [P1] Leads

VOS-052 [P1] Lead sources

VOS-053 [P1] Lead state transitions

VOS-054 [P1] Lead assignment

VOS-055 [P1] Minimal CRM dashboard

Milestone acceptance: a tenant can receive and manage a lead through the pre-AI pipeline with tenant isolation and audit coverage.

Gate E - Events and queues before AI

Complete VOS-012 and then VOS-080 before building the AI agent layer.

The sequence should be:

```
Business configuration
  -> CRM
  -> durable events
  -> queue execution
  -> conversations/provider abstraction
  -> AI provider abstraction
  -> tool framework
  -> workflow engine
  -> receptionist vertical slice
```

This prevents the LLM layer from becoming the orchestration layer.

Recommended next 10 PRs

1. Atomic refresh rotation + concurrency test.
2. Active-tenant refresh persistence + tests.
3. Membership revocation/session invalidation use case.
4. Auth rate limiting + proxy/origin security configuration.
5. Documentation reconciliation + ADR-014.
6. Service and ServiceArea schema migration + RLS tests.
7. BusinessProfile + Location application/API layer.
8. Services + ServiceAreas application/API layer.

9. Contacts CRUD + tenant tests.
10. Leads + lifecycle state machine + audit.

ADR-013 Amendment

ADR-013 — Session and Token Strategy

• Status: Accepted

• Date: 2026-09-19

Context

ADR-011 separates authentication, tenant membership and authorization, and SDD-09 §2 requires "short-lived access credentials and a secure refresh/session mechanism". Neither names a mechanism.

Two properties of this platform constrain the choice:

1. A user may belong to several tenants (FR-002), and ADR-011 notes that "multiple tenant memberships require explicit active-tenant handling". Whatever carries identity must also carry, or resolve, an active tenant.
2. The data is customer business information under RLS (ADR-005). Losing a credential must be recoverable — access has to be revocable without waiting for natural expiry.

Decision

Short-lived **JWT access tokens** plus **rotating opaque refresh tokens** persisted server-side in PostgreSQL.

Access token

• JWT, HS256, 15 minute lifetime.

• Claims: `sub` (user id), `tid` (active tenant id), `role`, `jti`, `iat`, `exp`.

• Verified from the signature alone — no database round-trip on the hot path.

• Capabilities are **not** embedded. They are derived from `role` at request time, so the token stays small and a capability-model change does not require reissuing tokens.

Refresh token

• Opaque, 256 bits of CSPRNG output. Never a JWT: there is nothing to read client-side, and opacity keeps it inert if leaked into a log.

• Stored **hashed** (SHA-256) in `refresh_tokens`. A database disclosure must not yield usable credentials.

• **Rotated on every use**: redeeming a refresh token issues a new one and marks the old one replaced.

• Tokens form a **family** (`family_id`). Redeeming an already-replaced token means it was captured, so the entire family is revoked — the

standard reuse-detection response.

Tenant switching

`POST /auth/tenant` verifies membership and issues a new access token with a different `tid`. Because access tokens are short-lived, the stale-tenant window is bounded by the access token lifetime.

Consequences

Positive:

• No session lookup on the common path; only refresh touches the database.
• Refresh tokens are revocable individually or per family, so a stolen credential can be killed without a global logout.
• Reuse detection gives a signal that a token was captured, not just that one expired.
• Sessions survive a Redis restart: Redis stays a queue backend and holds no durable business state (AGENTS.md §2.8).

Negative:

• `role` and `tid` in the access token can be up to 15 minutes stale. A role downgrade or a membership revocation is not instant. Accepted: the window is bounded and short, and the alternative is a database read per request. Operations needing immediate effect must revoke the refresh family **and** wait out the access token, or check membership explicitly.
• A signing secret must be managed (SDD-09 §7).
• Requires a table that SDD-04 §3 did not list — see below.

Schema change

This decision adds one table, `refresh_tokens`, to the set fixed by SDD-04 §3. It is a global, user-scoped table like `users`: it carries no `tenant_id` and no RLS policy, because a refresh token authenticates a person, not a tenant session, and it must be readable before any tenant is known. SDD-04 and SDD-09 are updated accordingly.

Rejected alternatives

Opaque sessions in Redis

Rejected because Redis is the queue backend and AGENTS.md §2.8 is explicit that it "is for asynchronous work, not durable business state". A Redis flush would log out every user, and session records would become state that matters living in a store treated as disposable.

JWT access + JWT refresh

Rejected because a self-contained refresh token cannot be revoked without a

denylist, which reintroduces the server-side lookup while losing the ability

to detect reuse.

Long-lived JWT only

Rejected: no revocation path at all.

V0.2 Amendment

Atomic rotation

Rotation is a compare-and-set operation, not a read-then-write operation. Exactly one concurrent redemption of a live token may claim it. A second redemption is treated according to the reuse-detection policy.

Active tenant continuity

A refresh session stores nullable `active_tenant_id`. Tenant switching re-verifies membership and updates the server-side session/family state. Successor refresh tokens inherit the validated active tenant. Refresh must revalidate that the membership remains active before issuing a tenant-scoped access token.

Revocation semantics

Membership revocation must revoke relevant refresh sessions/families in the same application-level security operation. Access JWTs retain the documented short stale window unless a route explicitly performs live membership validation.

ADR-014

ADR-014 - Service and Service Area Modeling

• Status: Accepted

• Date: 2026-09-21

Context

The first vertical is remodeling, where qualification and routing depend on both the requested service and whether the property is inside the contractor's service area. These concepts are also shared by HVAC, roofing, plumbing, dental, and other local-business verticals.

Storing them only as arbitrary JSON would weaken validation, querying, analytics, and AI tool grounding.

Decision

Create tenant-scoped relational entities:

```
Service
- id
- tenant_id
- key
- name
- description
- active
- metadata

ServiceArea
- id
- tenant_id
- type (CITY | POSTAL_CODE | RADIUS | REGION)
- value/configuration
- active
```

Both entities use RLS. Vertical templates seed default services and qualification mappings.

Consequences

Positive:

• reliable qualification;

• queryable service coverage;

• stronger AI grounding;

• reusable across verticals;

• analytics can group by service.

Negative:

• two additional tables and APIs;

• geospatial sophistication may require later schema evolution.

Deferred

Do not introduce PostGIS in V0.2 unless radius/polygon requirements prove that simple city/ZIP/radius configuration is insufficient.

SDD-03 Updated

SDD-03 — Domain Model

1. Core entities

Tenant

The security and billing boundary for a business account.

User

A human identity.

TenantUser

Membership connecting a user to a tenant and role.

BusinessProfile

Business-specific operational configuration.

Location

Physical/service location associated with a tenant.

Contact

A person or organization interacting with the business.

Lead

A sales opportunity associated with a contact.

Conversation

A channel-independent interaction thread.

Message

An inbound/outbound communication item.

Call

A telephony interaction and its metadata/transcript reference.

Appointment

A scheduled customer interaction.

Estimate

A sales estimate and its lifecycle.

Workflow

A declarative automation definition.

WorkflowRun

One execution instance of a workflow.

WorkflowStepRun

Execution state for an individual workflow step.

Agent

An AI capability definition.

AgentRun

One execution of an AI agent.

ToolExecution

One invocation of an agent tool.

KnowledgeDocument

A tenant-owned source document.

KnowledgeChunk

An embedded semantic chunk of a knowledge document.

Integration

Connection metadata for an external provider.

Event

A durable business event emitted by the platform.

AuditLog

A security/operations record of important actions.

2. Relationship model

```
Tenant
 ■■■ TenantUser ■■ User
 ■■■ BusinessProfile
 ■■■ Location
 ■■■ Contact
 ■    ■■■ Lead
 ■         ■■■ Conversation
 ■         ■    ■■■ Message
 ■         ■■■ Call
 ■         ■■■ Appointment
 ■         ■■■ Estimate
 ■■■ Workflow
 ■    ■■■ WorkflowRun
 ■         ■■■ WorkflowStepRun
 ■■■ Agent
 ■    ■■■ AgentRun
 ■         ■■■ ToolExecution
 ■■■ KnowledgeDocument
 ■    ■■■ KnowledgeChunk
 ■■■ Integration
 ■■■ AuditLog
```

3. Tenant-scoped rule

All entities except global catalog/configuration entities are tenant-scoped.

Every tenant-scoped entity must contain `tenant_id`.

4. Lifecycle state

Lead lifecycle is a business concept, not an AI concept.

AI may recommend or perform transitions through tools, but the domain owns

the valid state transitions.

5. Custom fields

Vertical-specific attributes are represented using:

• `CustomFieldDefinition`

• `CustomFieldValue`

Do not add vertical-specific columns to core entities unless the field becomes

a universal concept.

6. Vertical templates

Templates provide:

• services;

• custom fields;

• qualification questions;

• default workflows;

• prompt/context configuration;

• dashboard configuration.

Templates must not fork the core domain model.

V0.2 Addendum - Authentication session entity

Add `RefreshToken` as a global identity/session entity. It is not tenant-scoped because it authenticates a user before an active tenant is necessarily selected.

Required fields include `user_id`, token digest, family identifier, replacement/revocation metadata, expiry, diagnostics, and `active_tenant_id` (nullable). `active_tenant_id` is a session preference and MUST be revalidated against active membership before use.

Add tenant-scoped `Service` and `ServiceArea` entities before VOS-042/VOS-043. Vertical templates seed these records; they do not fork the core schema.

SDD-04 Updated

SDD-04 — Database Specification

1. PostgreSQL

PostgreSQL is the source of truth for durable business state.

2. Required extension

```
CREATE EXTENSION IF NOT EXISTS vector;
```

The exact embedding dimensions are provider/model configuration and must not be hardcoded into unrelated domain code.

3. Initial tables

```
tenants
users
tenant_users
refresh_tokens
business_profiles
locations

contacts
leads
lead_sources

conversations
messages
calls

appointments
estimates

workflows
workflow_runs
workflow_step_runs

agents
agent_runs
tool_executions

knowledge_documents
knowledge_chunks

integrations
events
audit_logs

custom_field_definitions
custom_field_values
```

Global tables

`users` and `refresh_tokens` are not tenant-scoped. Both describe a person rather than a tenant's data, and both must be readable before an active tenant is known — authentication looks a user up by email, and a refresh token is redeemed before any tenant is selected. Neither carries `tenant_id` and neither has an RLS policy (ADR-013).

Tenant-scoped access to people goes through `tenant_users`, which is protected.

4. Common columns

Tenant-scoped tables should normally contain:

```
id
tenant_id
created_at
updated_at
```

Where useful:

```
deleted_at
created_by
updated_by
version
```

5. IDs

Use UUID/UUIDv7-style IDs.

IDs exposed through APIs must not reveal tenant sequencing.

6. Indexing baseline

Every tenant-scoped high-volume table should have an index beginning with

`tenant_id` where queries commonly filter by tenant.

Examples:

```
leads(tenant_id, status)
leads(tenant_id, created_at)
contacts(tenant_id, phone)
conversations(tenant_id, updated_at)
appointments(tenant_id, starts_at)
workflow_runs(tenant_id, status)
audit_logs(tenant_id, created_at)
```

Exact indexes must be validated against query plans.

7. JSONB

Use JSONB for:

• flexible integration metadata;

• workflow configuration;

• provider-specific metadata;

• tenant configuration that is intentionally schema-flexible.

Do not use JSONB to avoid modeling stable business concepts.

8. Vector storage

`knowledge_chunks` should contain:

```
id
tenant_id
document_id
content
embedding
metadata
created_at
```

Metadata can include:

• document type;

• service;

• location;

• source;

• version.

9. Transactions

Use transactions for state transitions that must remain atomic.

Examples:

• create lead + initial event;

• create appointment + update lead status;

• workflow state + event emission.

For cross-system side effects, use an outbox-style pattern when required rather

than pretending the external provider participates in the database transaction.

10. Migrations

Production uses:

```
prisma migrate deploy
```

Never use:

```
prisma db push
```

against production.

11. Backup/recovery

The production database must have:

• automated backups;

• restore procedure;

• tested recovery process;

• documented retention policy.

Exact infrastructure values are environment decisions, not hardcoded in the domain.

V0.2 Addendum - Schema reconciliation

The V0.1 table inventory is no longer an exact closed list. ADR-approved schema evolution is allowed when the SDD is updated in the same change.

Additional approved tables:

```
refresh_tokens
services
service_areas
```

`refresh_tokens` is global. `services` and `service_areas` are tenant-scoped and MUST receive ENABLE + FORCE RLS and isolation tests.

The database package currently contains six migrations in the reviewed snapshot. Backlog documentation must not hardcode migration counts as completion evidence.

SDD-09 Updated

SDD-09 — Security Specification

1. Threat model

The platform handles:

• customer contact information;

• business information;

• conversations;

• appointments;

• estimates;

• integration credentials;

• AI-generated actions.

Primary threats:

• cross-tenant access;

• credential leakage;

• unauthorized AI actions;

• webhook spoofing;

• prompt injection;

• data leakage through logs;

• insecure background jobs.

2. Authentication

Use short-lived access credentials and a secure refresh/session mechanism.

Authentication identifies the user.

Authorization determines what the user can do.

The mechanism is specified in ADR-013: short-lived JWT access tokens (15 minutes, carrying the active tenant) plus rotating opaque refresh tokens stored hashed in PostgreSQL, with family-based reuse detection.

Passwords are stored as Argon2id digests, never reversibly.

3. Authorization

Permissions are scoped to:

• tenant;

• role;

• capability.

Example:

```
lead.read
lead.write
appointment.read
```

```
appointment.write
estimate.read
estimate.write
workflow.manage
integration.manage
audit.read
```

4. AI authorization

AI tools have their own permissions.

A user may have:

```
estimate.write
```

without granting an AI agent:

```
estimate.approve
```

5. Prompt injection

Retrieved/customer-provided text is untrusted input.

The system must not treat arbitrary customer content as system policy.

Business rules and tool permissions live outside the model's generated text.

6. Sensitive actions

Potentially high-impact actions require:

• explicit authorization;

• validation;

• audit;

• optional human approval.

Examples:

• changing estimate totals;

• refunds;

• deleting customer data;

• changing business rules.

7. Secrets

Secrets must be injected through environment/secret-management infrastructure.

Never commit:

• API keys;

• OAuth client secrets;

• database passwords;

• JWT secrets;

• provider tokens.

8. Logging

Do not log:

• raw credentials;

• access tokens;

• passwords;

• unnecessary full message bodies;

• unnecessary customer PII.

Use IDs and metadata wherever possible.

9. Data retention

Retention policies must be defined before production scale.

The system must support future deletion/anonymization workflows.

10. Audit

Audit state-changing actions involving:

• authentication;

• authorization;

• AI tools;

• integrations;

• workflow activation;

• customer/lead mutations;

• high-impact operations.

V0.2 Addendum - Authentication session requirements

Authentication uses short-lived JWT access tokens plus rotating opaque refresh tokens persisted as hashes in PostgreSQL.

Additional requirements:

• refresh-token consumption MUST be atomic under concurrent redemption;

• the server-side refresh session MUST preserve the selected active tenant for multi-tenant users, with membership revalidation;

• membership revocation MUST invalidate relevant refresh sessions as part of the security use case;

• login and refresh endpoints MUST have abuse/rate controls before production;

• production deployment MUST define trusted proxy, trusted origin, and CSRF behavior for cookie-authenticated endpoints;

• refresh tokens remain HttpOnly and Secure in production.

Technical Backlog Updated

Vautrin OS — Technical Backlog V0.1

Backlog rules

Priority:

• P0 = blocks MVP / security / architecture

• P1 = required for first production vertical slice

• P2 = useful MVP enhancement

• P3 = post-MVP

Status:

• TODO

• IN_PROGRESS

• BLOCKED

• DONE

Each task should be implemented as a small PR where practical.

EPIC E0 — Repository and developer experience

VOS-001 [P0] Initialize pnpm workspace

Status: DONE — pnpm workspace, pinned pnpm 9.15.4 + Node 22.19.0, clean install verified.

Acceptance criteria

• root package.json exists;

• pnpm-workspace.yaml exists;

• package manager version is pinned;

• Node version is pinned;

• install works from a clean checkout.

VOS-002 [P0] Configure Turborepo

Status: DONE — turbo.json defines dev/build/lint/typecheck/test with explicit dependsOn.

Acceptance criteria

• `dev`, `build`, `lint`, `typecheck`, `test` tasks are defined;

• task dependencies are explicit;

• affected tasks can be run locally.

VOS-003 [P0] Create apps/web

Status: DONE — Nuxt 4.5 + Vue 3 + TypeScript (strict), Nuxt 4 `app/` layout, runtime-config API base, same-origin Nitro proxy to apps/api, status page consuming @vautrin/contracts, component tests via @nuxt/test-utils.

Nuxt 4 + Vue 3 + TypeScript.

VOS-004 [P0] Create apps/api

Status: DONE — NestJS 12 (ESM) HTTP app: validated environment, global ValidationPipe, consistent error filter, OpenAPI, liveness/readiness probes, graceful shutdown.

NestJS application.

VOS-005 [P0] Create apps/worker

Status: DONE — NestJS 12 standalone context consuming the four ADR-009 BullMQ queues with bounded retries and job-scoped tenant context.

NestJS worker application.

VOS-006 [P0] Configure shared TypeScript standards

Status: DONE — tsconfig.base.json (strict, noUncheckedIndexedAccess, exactOptionalPropertyTypes); Biome 2.5.14 (biome.json) for TypeScript and Vue `<script>` blocks; ESLint 10 + eslint-plugin-vue (eslint.config.mjs) for Vue `<template>` blocks only. `pnpm lint` runs both.

Strict mode, path conventions, linting, formatting.

VOS-007 [P0] Add AGENTS.md

Status: DONE — AGENTS.md installed at the repository root.

Install repository engineering contract.

EPIC E1 — Local infrastructure

VOS-010 [P0] Docker Compose

Status: DONE — infra/docker-compose.yml — pgvector/pgvector:pg17 + redis:7, healthchecks, role bootstrap.

PostgreSQL + pgvector + Redis.

VOS-011 [P0] Database package

Status: DONE — packages/database — Prisma schema (26 tables per SDD-04 §3), 4 migrations, tenant-scoped client.

Prisma configuration, migrations, client.

VOS-012 [P0] Redis/BullMQ package

Shared queue configuration.

VOS-013 [P1] Local email/mock provider

Mailpit or equivalent for development.

EPIC E2 — CI/CD

VOS-020 [P0] GitHub Actions PR pipeline

Status: DONE — .github/workflows/ci.yml `quality` job: lint (Biome + ESLint), format check, typecheck, unit tests, build.

Lint, typecheck, unit tests, build.

VOS-021 [P0] Integration-test service containers

Status: DONE — ci.yml `integration` job with pgvector/pg17 + redis:7 service containers, role provisioning matching infra/postgres/initdb, migrations, and an assertion that the runtime role cannot bypass RLS.

PostgreSQL + Redis.

VOS-022 [P1] E2E pipeline

Run critical web/API tests.

VOS-023 [P1] Container builds

Status: DONE — Dockerfile per deployable (apps/api, apps/worker, apps/web) plus a separate migration-runner image (packages/database) per SDD-12 §7, and a root docker-compose.yml running the full stack.

Build web/api/worker images.

VOS-024 [P1] Staging deployment

Automated deployment after main.

VOS-025 [P1] Production deployment gate

Approval + smoke tests + rollback documentation.

VOS-026 [P0] Secret scanning

Status: DONE — ci.yml `security` job: TruffleHog secret scan over full history, `pnpm audit` gated at high, non-root Dockerfile assertion, and scripts/check-rls-coverage.mjs. Plus .github/workflows/codeql.yml (security-extended).

Prevent accidental credential commits.

EPIC E3 — Authentication and tenant foundation

VOS-030 [P0] User authentication

Status: DONE — POST /auth/login, /auth/refresh, /auth/logout, GET /auth/me. Argon2id passwords, JWT access tokens, rotating refresh tokens with reuse detection (ADR-013).

Login/session flow.

VOS-031 [P0] Tenant entity

Status: DONE — TenantService.provision creates a tenant and its first owner in one transaction, without any RLS bypass.

Tenant creation and lifecycle.

VOS-032 [P0] Tenant membership

Status: DONE — TenantService.addMember / revokeMembership over tenant_users, with role assignment.

User ↔ tenant roles.

VOS-033 [P0] Tenant context

Status: DONE — JwtAuthGuard resolves the active tenant from the access token; POST /auth/tenant switches it after re-verifying membership. Client-supplied tenant ids are never trusted (SDD-05 §2).

Resolve active tenant from authenticated request.

VOS-034 [P0] RBAC

Status: DONE — capability model in @vautrin/auth (ADR-011, SDD-09 §3), enforced by a global CapabilityGuard with @RequireCapability. Denials are audited.

Owner/admin/manager/staff/read-only.

VOS-035 [P0] RLS framework

Status: DONE — ENABLE + FORCE RLS on all 25 tenant-scoped tables, app.current_tenant_id() helper, drift guard fails the migration if a tenant_id table is left unprotected.

Enable and test PostgreSQL RLS.

VOS-036 [P0] Cross-tenant security tests

Status: DONE — packages/database/test/tenant-isolation.integration.test.ts — SELECT/INSERT/UPDATE/DELETE, conversation, workflow, vector, and background-job isolation, run as the non-privileged runtime role.

Read/write/delete isolation.

EPIC E4 — Business configuration

VOS-040 [P1] Business profile

Name, contact details, timezone, locale.

VOS-041 [P1] Locations

Service/physical locations.

VOS-042 [P1] Services

Service catalog.

VOS-043 [P1] Service areas

Cities/ZIP/service regions.

VOS-044 [P1] Business hours

Weekly availability rules.

VOS-045 [P1] Custom field definitions

Generic vertical-specific fields.

VOS-046 [P1] Custom field values

Lead/contact/estimate values.

EPIC E5 — CRM

VOS-050 [P1] Contacts

CRUD + search + pagination.

VOS-051 [P1] Leads

CRUD + pipeline.

VOS-052 [P1] Lead sources

Google, website, phone, referral, ads, etc.

VOS-053 [P1] Lead state transitions

Validate lifecycle transitions.

VOS-054 [P1] Lead assignment

Owner/staff assignment.

VOS-055 [P1] CRM dashboard

Lead counts and pipeline.

EPIC E6 — Conversations

VOS-060 [P1] Conversation entity

Channel-independent thread.

VOS-061 [P1] Message entity

Inbound/outbound state.

VOS-062 [P1] Messaging provider interface

No provider-specific domain dependency.

VOS-063 [P1] First messaging provider adapter

Select provider during implementation.

VOS-064 [P1] Webhook ingestion

Signature validation + deduplication.

VOS-065 [P1] Unified inbox API

List/view/respond.

EPIC E7 — AI foundation

VOS-070 [P1] LLMProvider interface

Generation/streaming contract.

VOS-071 [P1] First LLM adapter

Select provider during implementation.

VOS-072 [P1] Agent entity/configuration

Versioned agent configuration.

VOS-073 [P1] Tool framework

Typed tool contracts + authorization.

VOS-074 [P1] Business context builder

Structured tenant context.

VOS-075 [P1] Knowledge document model

Tenant knowledge.

VOS-076 [P1] Embedding pipeline

Chunk + embed + persist.

VOS-077 [P1] pgvector retrieval

Tenant-filtered semantic retrieval.

VOS-078 [P1] Agent execution telemetry

Agent/tool/provider metadata.

EPIC E8 — Workflow engine

VOS-080 [P0] Event model

Versioned internal event schema.

VOS-081 [P1] Workflow definition model

Versioned workflows.

VOS-082 [P1] Workflow run model

Persist execution state.

VOS-083 [P1] Workflow worker

BullMQ execution.

VOS-084 [P1] Wait step

Delayed jobs.

VOS-085 [P1] Condition step

Safe condition evaluation.

VOS-086 [P1] Message action

Send through provider abstraction.

VOS-087 [P1] Agent action

Invoke agent.

VOS-088 [P1] Idempotency

Prevent duplicate side effects.

VOS-089 [P1] Retry/failure policy

Bounded retries + escalation.

EPIC E9 — Appointments

VOS-090 [P1] Calendar provider interface

Availability/create/cancel.

VOS-091 [P1] Calendar adapter

First provider.

VOS-092 [P1] Appointment domain

State and validation.

VOS-093 [P1] Booking tool

AI-authorized appointment creation.

VOS-094 [P1] Booking workflow

Lead → qualification → booking.

EPIC E10 — AI Receptionist

VOS-100 [P1] Receptionist agent

Business-grounded conversation.

VOS-101 [P1] Qualification toolset

Service/location/budget/timeline.

VOS-102 [P1] Human handoff

Explicit escalation.

VOS-103 [P1] Receptionist evaluation set

Synthetic regression cases.

VOS-104 [P1] Missed-call workflow

Call missed → AI/SMS → follow-up.

EPIC E11 — Estimates

VOS-110 [P1] Estimate entity

Create/send/status.

VOS-111 [P1] Estimate dashboard

Pending/sent/follow-up.

VOS-112 [P1] Estimate follow-up workflow

Automated reminders.

VOS-113 [P2] Estimate AI assistant

Draft communication only; no autonomous price approval.

EPIC E12 — Audit and observability

VOS-120 [P0] Structured logging

Request/job correlation.

VOS-121 [P0] Audit log

Status: PARTIAL — AuditService writes audit_logs for authentication and authorization-denied events. Customer/lead mutations and integration actions are added as those modules land.

Important state changes.

VOS-122 [P1] Workflow telemetry

Run/step metrics.

VOS-123 [P1] AI telemetry

Provider/model/token/latency where available.

VOS-124 [P1] Error tracking

Centralized exception reporting.

VOS-125 [P1] Operational dashboard

System health and workflow metrics.

EPIC E13 — Remodeling template

VOS-130 [P1] Remodeling services

Kitchen, bathroom, whole-home, additions, etc.

VOS-131 [P1] Remodeling custom fields

Project type, property type, budget, timeline, location.

VOS-132 [P1] Remodeling qualification workflow

VOS-133 [P1] Remodeling default receptionist configuration

VOS-134 [P1] Remodeling seed/demo tenant

EPIC E14 — End-to-end vertical slice

VOS-140 [P0] Critical path E2E

```
Create tenant
→ configure remodeling
→ inbound lead
→ contact
→ lead
→ conversation
→ AI qualification
→ follow-up
→ appointment
→ estimate
→ estimate follow-up
→ audit
```

VOS-141 [P0] Tenant isolation E2E

VOS-142 [P1] Provider failure E2E

Messaging/calendar failure and recovery.

VOS-143 [P1] Worker retry E2E

Ensure idempotent external action.

Security review log

2026-09-21 — authentication CI blockers

• Generate and mask an ephemeral integration-test JWT key; forward it only to uncached Turbo integration tasks, without a production/repository secret.
• Keep Semgrep blocking, with documented call-site exceptions for the 11 reviewed global identity/session and tenant-bootstrap accesses.
• Add configuration and security-rule regressions; leave the existing private-repository CodeQL skip unchanged.
• Local validation: lint/typecheck/build, 70 unit tests, and 52 integration tests pass. Integration tests also pass from a source snapshot without `.env`, using the workflow-generated key. Full Semgrep scan has zero findings; both project-rule regression suites pass. GitHub confirmation requires committing/pushing these changes.

2026-09-19 — first security pass (CI introduction)

Findings and disposition:

#	Finding	Severity	Status
1	Nuxt `/api` proxy allowed path traversal out of the `/api` prefix, reaching arbitrary paths on the internal API host	High	Fixed — path confinement + 12 regression tests
2	`deepmerge-ts@7.1.5` stack exhaustion (GHSA-ggr8-5vv4-36mx) via `@prisma/client`	High	Fixed — pnpm override to >=8.0.2
3	`x-correlation-id` reflected unbounded and unvalidated into responses and future logs	Medium	Fixed — UUID validation, invalid values dropped
4	No security headers on API or web; `X-Powered-By` disclosed Express and Nuxt	Medium	Fixed — helmet on API, CSP/nosniff/frame-ancestors on web
5	Requests outside the API global prefix bypass the exception filter and return an Express HTML error page	Low	Accepted — discloses only that Express is in use; revisit with VOS-120
6	Compose files default to a well-known local password	Low	Accepted — local dev tooling; production provisioning is VOS-024/VOS-025

See SECURITY.md for the standing control list and known gaps.

Definition of MVP complete

V0.1 is complete only when:

• the critical vertical slice works end-to-end;
• tenant isolation is verified;

• CI/CD deploys repeatably;

• AI tool permissions are enforced;

• workflows are retryable/idempotent;

• a remodeling template provisions a usable tenant;

• the first real customer can operate the revenue workflow without engineering intervention for normal cases.

Post-MVP backlog

P2/P3 candidates:

• review automation;

• customer re-engagement;

• AI proposal drafting;

• project/task management;

• invoicing;

• payments;

• advanced analytics;

• multi-location support;

• workflow builder UI;

• additional home-service templates;

• additional communication providers;

• enterprise tenant isolation options.

V0.2 Review Addendum

Immediate correction tasks

VOS-027 [P1] Authentication abuse controls

Status: TODO

VOS-028 [P1] Cookie/origin/proxy security contract

Status: TODO

VOS-037 [P0] Atomic refresh-token rotation

Status: TODO

VOS-038 [P0] Preserve active tenant across refresh

Status: TODO

VOS-039 [P0] Secure membership revocation use case

Status: TODO

VOS-014 [P0] Documentation consistency gate

Status: TODO

Status corrections

• VOS-121 Audit log: **IN_PROGRESS**. Authentication/authorization coverage exists; remaining mutation/tool/integration/workflow audit surfaces are pending.

• VOS-012 Redis/BullMQ package: implementation exists in API/worker queue infrastructure, but completion should be marked DONE only after its package-level acceptance criteria are explicitly verified.

• VOS-022 E2E pipeline: authentication integration/E2E coverage exists, but the product critical-path E2E pipeline remains TODO.

Next milestone order

Do not begin the AI/workflow product layer until VOS-037, VOS-038, VOS-039 and VOS-014 are complete. Then execute E4 Business Configuration followed by E5 CRM.